

Power BI Saved Slicers — Function Reference

Every function in the codebase, what it carries out, and where it is used.

Files

Three files matter: `app.py` (server), `templates/index.html` (page + JS), `saved_filters.json` (storage).

Server functions (app.py)

Storage

Function	Where it lives	What it does (carries out)
<code>_load_all_filters()</code>	<code>app.py</code>	Reads the JSON file from disk. Migrates any old-shape entries (where data sat directly under a user) into a <code>"_legacy_unscoped"</code> sub-bucket so nothing is lost when the new per-report shape kicks in.
<code>_save_all_filters(data)</code>	<code>app.py</code>	Writes the whole store back to disk atomically (temp file then rename).
<code>current_report_key()</code>	<code>app.py</code>	Builds the bucket key for the report currently embedded: <code>"groupId::reportId"</code> . Used by every save/list/delete endpoint so different reports never see each other's saves.

Identity (POC)

Function	Where it lives	What it does (carries out)
<code>get_user_id()</code>	<code>app.py</code>	Reads the current customer ID out of the Flask session. Returns <code>None</code> if no one is signed in. This is the single swap point to replace with real auth.
<code>POST /login</code>	<code>app.py</code>	Stores the customer's typed name/email in the session.
<code>GET/POST /logout</code>	<code>app.py</code>	Clears the session.
<code>GET /whoami</code>	<code>app.py</code>	Returns the current customer ID and the report bucket — used by the frontend for debug.

Power BI embed plumbing

Function	Where it lives	What it does (carries out)
<code>get_config()</code>	<code>app.py</code>	Returns the active Power BI config (tenant, client ID/secret, group, report). Falls back to <code>DEFAULT_CONFIG</code> .
<code>get_access_token()</code>	<code>app.py</code>	Calls Azure AD with client credentials to get a Power BI service token.
<code>get_embed_token()</code>	<code>app.py</code>	Asks Power BI for an embed token for the configured report, plus the embed URL.
<code>GET /</code>	<code>app.py</code>	Renders the embedded report page (<code>templates/index.html</code>), passing the embed token, report IDs, and current user ID into the template.

Saved-views API

Function	Where it lives	What it does (carries out)
<code>GET /api/filters</code>	<code>app.py</code>	Lists saved views for the current user, scoped to the current report bucket.
<code>POST /api/filters</code>	<code>app.py</code>	Saves a named view {filters, slicers}. Overwrites if the same name already exists for this user + report.
<code>DELETE /api/filters/<id></code>	<code>app.py</code>	Removes one saved view by ID for the current user + report.

Frontend functions (templates/index.html)

Identity & UI helpers

Function	Where it lives	What it does (carries out)
setMsg(text, isError)	templates/index.html	Writes a status line under the Saved Filters panel (green or red depending on success).
updateArmedBadge()	templates/index.html	Refreshes the small badge that says how many pages of the armed saved view have been applied and how many will be cleared on next visit.
pagesInArmed()	templates/index.html	Returns the unique page names contained in the armed saved view.

Page / visual plumbing (around the Power BI client)

Function	Where it lives	What it does (carries out)
wait(ms)	templates/index.html	Tiny promise-based delay so we wait for a page to finish loading visuals after activation.
getActivePageName()	templates/index.html	Wraps report.getPages() and returns the name of the page in front of the customer.
activatePageByName(name)	templates/index.html	Calls page.setActive() if needed, then waits ~400 ms. Used during the initial load to seed the cache.

Capture (Save side)

Function	Where it lives	What it does (carries out)
refreshCacheForActivePage()	templates/index.html	Reads slicer state on the currently active page via visual.getSlicerState() for every slicer visual that has a selection, and stores the result in slicerCache[pageName]. Called on every pageChanged event and right before save.
captureSlicers()	templates/index.html	Refreshes the cache for the active page, then returns the flat list of all slicers in slicerCache (all visited pages). This is what Save sends to the server.
saveCurrentFilters()	templates/index.html	Reads the name field, calls report.getFilters() for any report-level filters, calls captureSlicers() for slicer state, POSTs to /api/filters, then refreshes the saved-views list.

Apply (Apply side)

Function	Where it lives	What it does (carries out)
sanitizeFilters(filters)	templates/index.html	Strips read-only fields from filters captured by getFilters so setFilters does not reject them on the way back in.
normalizeSaved(saved)	templates/index.html	Accepts either the new {filters, slicers} object shape or an old bare list, and returns a consistent {filters, slicers} object.
clearSlicersOnActivePage()	templates/index.html	For every slicer on the active page, calls setSlicerState({filters: []}) to wipe the selection. Used at the start of an Apply to clear stale clicks before the saved values are pushed.
applyArmedForPage(pageName)	templates/index.html	For each saved-set slicer that belongs to pageName, calls visual.setSlicerState() with the saved state. Marks the page as applied so it is not re-pushed.
applyFilters(saved)	templates/index.html	Main Apply handler. Pushes report-level filters via setFilters/updateFilters, arms the saved set in memory, computes pagesToClear, clears the active page, applies the active page's slicers, and updates the badge. Other pages wait for pageChanged.

Saved-views list UI

Function	Where it lives	What it does (carries out)
----------	----------------	----------------------------

loadSavedList()	templates/index.html	GETs /api/filters and renders one row per saved view with name, counts, an Apply button and a Delete button. Wires both buttons to the right handlers.
deleteFilter(id)	templates/index.html	DELETES /api/filters/<id> and re-renders the list.

Power BI events wired up

Function	Where it lives	What it does (carries out)
report.on('loaded')	templates/index.html	Caches the initial active page and auto-applies the most-recent saved view if one was queued at login.
report.on('rendered')	templates/index.html	Sets the status banner to "rendered".
report.on('error')	templates/index.html	Surfaces Power BI errors to the status banner.
report.on('pageChanged')	templates/index.html	On every page change: marks the page visited, runs clearSlicersOnActivePage if the page was queued for clearing, refreshes the cache for the new page, then runs applyArmedForPage to push any waiting saved-view slicers.

Power BI client APIs touched (the only ones)

Function	Where it lives	What it does (carries out)
report.getFilters() / setFilters() / updateFilters()	frontend	Report-level filters from the right-hand Filters pane. setFilters replaces; updateFilters("Replace") is the fallback.
report.getPages()	frontend	Lists pages, each with .name, .displayName, .isActive, .setActive(), .getVisuals().
page.setActive() / page.getVisuals()	frontend	Makes a page active and lists visuals on it. getVisuals only returns visuals on the currently loaded page.
visual.getSlicerState() / setSlicerState()	frontend	Reads and writes one slicer's selection. Empty {filters:[]} clears the slicer. Only works on the active page.
report.on('loaded' 'rendered' 'error' 'pageChanged')	frontend	Lifecycle and navigation events. pageChanged is what drives lazy capture and lazy apply.